My newsletter promised one-click unsubscribe and answered every one with a 400

Emil Lindfors | August 28, 2026

I was looking at telemetry for something else when I noticed a run of 400s on `/api/unsubscribe`. Not many. Enough to remind me that I built a newsletter in February and then stopped thinking about it.

The 400s were mail clients doing exactly what my own headers had told them to do. Every newsletter I sent advertised `List-Unsubscribe-Post: List-Unsubscribe=One-Click`. An RFC 8058 client responds to that by POSTing the fixed string `List-Unsubscribe=One-Click` as `application/x-www-form-urlencoded`. My handler called `req.json()`, a form body is not JSON, and it fell straight through to the 400 branch. So every reader who pressed the native unsubscribe button in Gmail got a refusal from a message that had just promised them one click.

Advertising one-click support and then answering 400 is worse than never advertising it, which is a sentence I would have found more alarming if the list had more than one member on it.

The one address on the list

I should say what state this site is in, because the numbers in this post make no sense otherwise. The blog is not launched. I have not announced it anywhere, not submitted the feed to any aggregator, not put the link in front of anyone. The subscribe form has been sitting on a site I am deliberately not pointing people at while I finish building the parts underneath it, and the newsletter is one of those parts. An empty subscriber list is what I expected to find.

It was not quite empty. There was one address on it, which I did not recognise and had certainly not typed myself – a scraped or automated signup of the kind that finds any form left open long enough. Nothing had ever been sent to it, so no harm landed, but `/api/subscribe` added any address anyone typed, which made the form an open invitation to point a self-hosted mail server at strangers.

That matters more here than it would on Mailchimp. Sender reputation on a personal VPS is fragile in a way theirs is not, and one spam-trap hit from `postmaster@lindfors.no` costs more than this newsletter is worth. GDPR also wants demonstrable consent, and a form submission from an unverified address is not that.

Having nobody to inconvenience turned out to be the best possible time to take the whole thing apart.

Double opt-in without a database

The claim in the original post was that Stalwart's mailing list removes the need for a database entirely. Double opt-in is normally where a claim like that dies. You need somewhere to keep addresses that have asked to subscribe but have not confirmed yet, and that is a table, with a schema and a cleanup job for the ones that never confirm.

Except that the pending state is only ever `(address, deadline)`. Both of those can travel in the confirmation link itself, provided the server can tell its own links from forged ones. That is what an HMAC is for.

```
fn confirm_signature(secret: &str, email: &str, exp: u64) -> String {
    sign(secret, &format!("confirm:v1:{}:{}", exp, email))
}
```

`/api/subscribe` validates the address, checks the rate limiters, and mails a link. It does not touch the list at all. `/api/confirm` recomputes the signature, checks it and the expiry, and only then calls `x:MailingList/set`. Self-expiring at 48 hours, no table, no cleanup job, no schema, and the click is itself the demonstrable consent.

Two details in that one-line function are load-bearing.

`exp` leads the signed string, and the order matters. It is decimal digits terminated by a colon, so no other way of splitting those bytes leaves a well-formed `(expiry, address)` pair. The other way round, `exp=1` with `email=":2:a@b.com"` and `exp=12` with `email="a@b.com"` sign identically.

Both payloads also carry a purpose prefix, `confirm:v1:` and `unsub:v1:`, because one secret now signs two kinds of token. Without the prefix, a link proving that someone wants *in* would be a valid instruction to take them *out*.

Why the confirmation is a POST

`GET /api/confirm` renders a page with a button and performs nothing.

Links in email get fetched by things that are not the recipient: Outlook Safe Links, corporate scanners, whatever the receiving provider runs on the way in. A GET that subscribed would let any of those confirm on the reader's behalf, which defeats the mechanism entirely – you would have built a slower single opt-in. It also restores the ordinary rule that a GET is safe to repeat. The form is plain HTML with hidden fields, so it needs no JavaScript and nothing new in the CSP.

`/api/unsubscribe` works the same way, for a sharper reason: that URL sits in the footer of every newsletter I send, and a GET that unsubscribed would let a link scanner empty the list one reader at a time. Nobody notices they have been unsubscribed. They wonder why the mail stopped.

One more decision in the same area. Subscribing an address that is already on the list gets a byte-identical response to subscribing a new one. Saying "you're already subscribed" would turn the endpoint into a membership oracle for any address a stranger cares to type, and the uniform answer costs nothing: setting `recipients/<addr>` to `true` a second time is a no-op, so I never have to read the list to decide what to say.

What the rate limiter actually stops

Double opt-in makes one abuse case worse before it makes it better. The endpoint used to add a row; now it makes my server send mail to any address a stranger types. Unrated, that is a mail bomb anyone can aim at anyone. So the rate limiting shipped in the same change rather than after it: three Cloudflare bindings, keyed per IP and per address on subscribe, per IP on everything else.

Then I measured them against production instead of assuming.

- 60 parallel requests against a 15-per-60s limit: **41 came back 429**.
- 18 *sequential* requests against the same limit: **zero**.

The second number is the one worth having. The binding does not merely let through callers pacing themselves under the limit, it under-counts slow traffic comfortably over it, because the counters are per-location and eventually consistent across edge instances. It stops a burst and nothing else. A patient attacker at one request per second is not rate limited in any meaningful sense.

That promotes the WAF rule from nice-to-have to the actual defence. It is still not set up.

The bindings do fail closed, at least. A missing or erroring limiter returns 503 rather than carrying on unlimited, because a config slip that silently unprotects a mail sender is the exact failure the limiter exists to prevent.

One-click unsubscribe, and the end of fan-out

Back to the 400s. The obvious fix is to read the body as text and parse it as a form when it is not JSON. I wrote that, and then noticed it would have made things worse.

The one-click POST body is the fixed string `List-Unsubscribe=One-Click` and it identifies nobody. The recipient has to come from the URL. But under fan-out there is no per-recipient URL to put in the header: one message goes to `newsletter@lindfors.no` and Stalwart delivers byte-identical headers to everyone on the list. Accepting the form encoding on

its own would have turned a 400 into a 200 that still could not unsubscribe anyone, and a 200 is worse, because it looks fixed.

One-click is architecturally incompatible with fan-out. The header and the delivery model cannot both stay, and Gmail and Yahoo's bulk sender rules care about the header.

So the Worker now reads the list and sends one message per recipient, each carrying `HMAC(secret, "unsub:v1:<email>")` in both the header and the footer link:

```
for email in &recipients {
    let unsubscribe_url =
        unsubscribe_link(&site_url, email, &unsubscribe_signature(&secret,
email));
    let html = email_template(/* ... */, &unsubscribe_url);

    match jmap_send_email(&sender, email, &subject, &html,
Some(&unsubscribe_url)).await {
        Ok(()) => sent += 1,
        Err(e) => {
            console_error!("Newsletter send to {} failed: {}", email, e);
            failed.push(email.clone());
        }
    }
}
```

Unsubscribe tokens never expire, unlike the 48-hour confirmation ones. A confirmation link is an invitation and should go stale; an unsubscribe link sits in a mailbox for as long as the subscriber keeps the message, and a dead one is how a reader who wanted to leave reports you as spam instead. The token is an identifier rather than an authorisation. `/api/unsubscribe` is deliberately open, and the signature grants nothing that a bare JSON POST does not.

What sending one message each costs

Fan-out was cheaper and simpler and I am not going to pretend otherwise. What it bought: working one-click, a footer link that does not make the reader type their own address into a form, and per-recipient personalisation if I ever want it. What it cost:

- **A hard ceiling.** Every message is a subrequest, and Workers caps those per invocation at 50 on the free plan, one of which is already spent fetching the markdown. Sends above 45 recipients are refused rather than truncated, because mailing the first 45 of a longer list and reporting success is the worst available outcome.
- **A send that can half-succeed.** The endpoint returns `{"sent": N, "failed": [...]}` and answers 502 if any recipient failed, so a partial send needs a human rather than reading as success.

- **Idempotency got harder**, which is worth saying plainly. Under fan-out a retry double-sent to everyone, which is obviously wrong and therefore obviously avoided. Now a retry after a partial failure double-sends only to the recipients who already succeeded, which is subtler and much easier to do by accident. A real sent-marker is still not built.

The DKIM chase

With all of that shipped, Gmail still showed no unsubscribe control. I went through two wrong ideas before landing on the real one.

The first was that the headers never reached the wire. Plausible enough: the Worker sets them through JMAP as `header:List-Unsubscribe:asRaw`, `Email/set` succeeds whether or not Stalwart honours arbitrary raw header properties, and I had never checked which. Wrong. The delivered message carried both, correctly formed.

The second was that authentication was failing. Also wrong: `spf=pass`, `dmarc=pass`, `dkim=pass`.

The cause was inside the signature. Both DKIM signatures carried:

```
h=Date:Message-ID:Subject:From:To
```

RFC 8058 §4 requires the `List-Unsubscribe` and `List-Unsubscribe-Post` headers to be covered by the signature and named in the `h=` tag. Neither was. Gmail was behaving correctly by refusing: an unsigned unsubscribe URL is one an intermediary could have rewritten, so honouring it would let anything that touched the message on the way unsubscribe the recipient or redirect the POST.

It is a Stalwart config setting rather than code. `DkimSignature.headers` defaults to exactly those five headers, none of which are the two that matter here. Adding them fixed the `h=`, with no restart needed. Which means the original post was wrong twice in one sentence when it said those headers were important for deliverability. They are important. Mine had been unsigned and unusable for the entire life of that post.

And then Gmail still did not show the control, because it gates that on being a bulk sender, roughly 5,000 messages per 24 hours to gmail.com addresses. My list has one member, and I did not put it there.

So one-click is implemented, correct, and dormant. Everything under my control is verified: the endpoint honours the exact RFC 8058 request, replayed with a real production-minted token for a 200, the headers are present and signed, authentication passes. What is left is a variable I do not control. That is a better place to stop than pretending it works, and if you are about to walk down this path it is the thing worth knowing before you start.

Two things in the headers

The `Message-ID` was `<...@localhost>`. Stalwart builds messages with its `mail-builder` crate, which reads `gethostname()` and falls back to the literal string `localhost`; its own `serverHostname` setting does not feed that path. Fixed by setting the system hostname. A `localhost` Message-ID is a well-known spam-filter smell and it had been on every newsletter I had ever sent.

The Ed25519 DKIM signature reports `dkim=neutral (no key)` at Gmail and always will. The record is published and valid. Gmail does not implement RFC 8463, M365 fails it, Yahoo perm-fails it, and roughly half of providers validate it. Harmless, because DMARC passes on the RSA signature. Do not chase it.

I chased it. Before I knew the real selector names I probed eight guessed ones, found nothing, and wrote down that no record was published. When the message headers later handed me `202412e`, I carried the earlier miss forward instead of querying the name I now had, and the record had been sitting there the whole time. The failure was not the wrong conclusion. It was not re-testing once the input changed.

Bugs in passing

`routes` in `wrangler.toml` had never worked. TOML assigns a bare key to the most recently opened table, and `routes = [...]` sat at the end of the file below `[vars]`, so it was an environment variable *named* `routes` rather than a route declaration. Nothing warned, because `[vars]` accepts any key. The live route came from the dashboard and the config block asserting it was decorative. It surfaced only when I added `[[ratelimits]]`: those tables have a fixed schema, so the same misplaced key finally produced `Unexpected fields found in ratelimits[2] field: "routes"`. Moving it above the first table header made `wrangler deploy` print a trigger line it had never printed before.

`site-tools newsletter send` reported failures as success. It shells out to `curl -s`, which exits 0 on an HTTP 500, and the CLI only checked curl's exit status. Now `--fail-with-body`. This mattered a lot more the moment a send could partially fail.

`is_valid_email` accepted `example-.com`. The domain was checked only as a whole, so a trailing hyphen was caught on the last label and nowhere else, and the local part had no character restrictions at all, so a space or a quote went through. Found by a test rather than by reading. It matters more than it used to: the check used to gate a row in a list, and now it gates a message leaving the server, where an unresolvable domain comes back as a bounce.

The Worker had zero tests. It now has 30, covering token signing, expiry boundaries, the confirm/unsubscribe domain separation in both directions, plus-addressing round-trips, and email validation. `crate-type` gained `rlib` so `cargo test` links for the host target; `worker-build` only looks for the cdylib, so the deployed artifact is unchanged. The clock is a parameter in `check_confirm_token_at` because `Date::now` is a JS call that panics outside a runtime, and the expiry boundary is exactly the part worth testing.

The rotation trap

One thing here can silently undo the rest of it. Stalwart 0.16 added DKIM lifecycle management, and with `dkimManagement` set to Automatic it regenerates `DkimSignature` objects every 90 days, and new objects get the default `headers` map. The List-Unsubscribe coverage would vanish at the next rotation, one-click would break again, and nothing in the logs would say why. Mine is on Manual with selectors dating from December 2024, so it has never rotated. Re-check the `h=` after any rotation.

Two operational facts about 0.16 worth not rediscovering. The config lives in RocksDB rather than on disk, so `/opt/stalwart/etc/config.toml` is dead weight and editing it does nothing, including its `server.hostname`. And the REST management API is gone, so subscriber management is JMAP, the WebUI, or `stalwart-cli apply`.

Where this leaves it

Something like a day of work, on a system with no real subscribers to break, which was the point of doing it now rather than later.

The WAF rule is the highest-value thing left, because the Workers binding only stops bursts and `/api/subscribe` sends mail on demand. After that: sent-markers so a retry cannot double-send, a staging list, splitting the admin key into a send-only token, and `List-Id`, which is correct on its own merits but will not flip the Gmail control and might move the newsletter from Primary to Promotions.

Then I will invite a few friends and see what breaks. All of the above was verified against a list of one, and a list of one is not the same as knowing it works. The worker is on GitHub if you want to read it, and if you ever get to the point where Gmail shows you an unsubscribe button on your own mail, I would like to hear how many messages a day it took.